Pragmatic Functional Programming

with TypeScript - Day 2

Olarn U. - Odds Team

Table of Contents

Topic covered

Day 1

- Introduction to FP
- FP Syntax in TypeScript
- Functions
- Functions Composition
- Types
- Higher-Order Function
- Currying
- Recursive

Day 2

- Option, Some, None
- Either, Left, Right
- ADT, Pattern Matching
- Kind, Higher-kinded Types
- Type Classes
- IO, IOEither
- Task, TaskEither

Day 3

- Category Theory
- Magma, Semigroup, Monoid
- Functor
- Applicative Functor
- Monads

Option

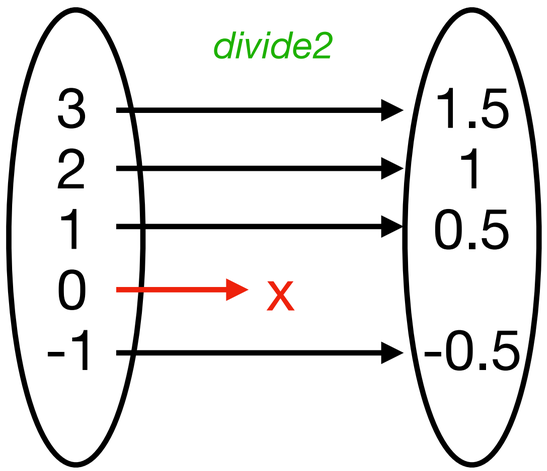
Non-Deterministic Function Handling

Non-Deterministic (Partial) Function

The function that accept only certain of value of the input type universe

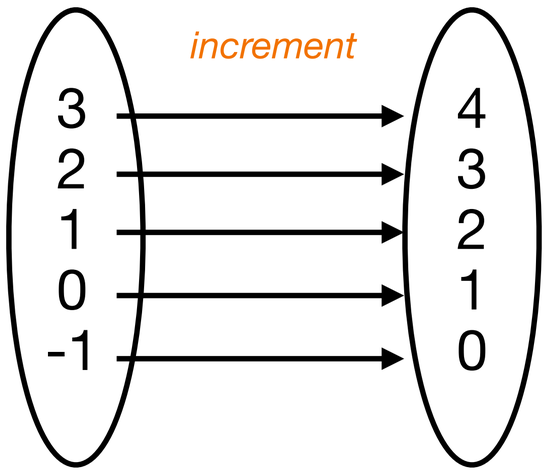
Non-Deterministic Function

```
function divide2(x: number) {
  return 2 / x
}
```



Deterministic Function

```
function increment(x: number) {
  return x + 1
}
```



Function in FP should be total, but how to handle this?

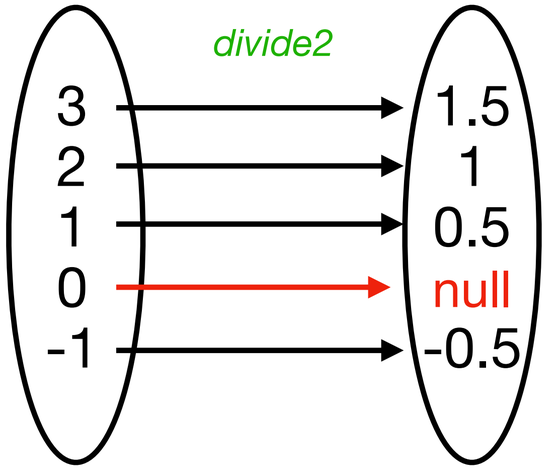
Let's return something indicate output is not available for 0.

Null?

That means we need to consider null (undefined in TS) everywhere possible…

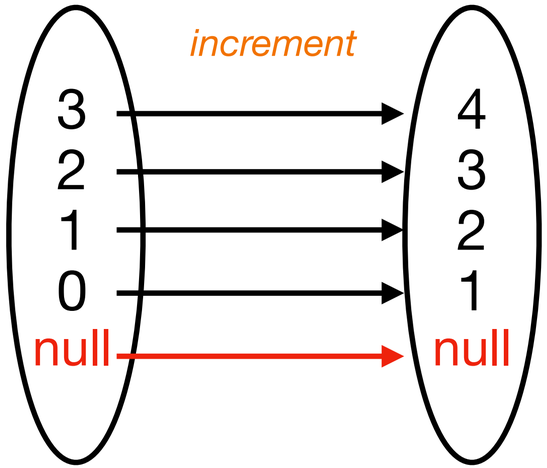
Non-Deterministic Function

```
function divide2(x: number): number | undefined {
  if (x === 0) {
     return undefined
  }
  return 2 / x
}
```



Deterministic Function

```
function increment(x: number): number | undefined {
  if (x < 0) {
     return undefined
  }
  return x + 1
}
```



Missing considering the null value can lead to unexpected behaviour and leads to bug.

Do you ever heard…

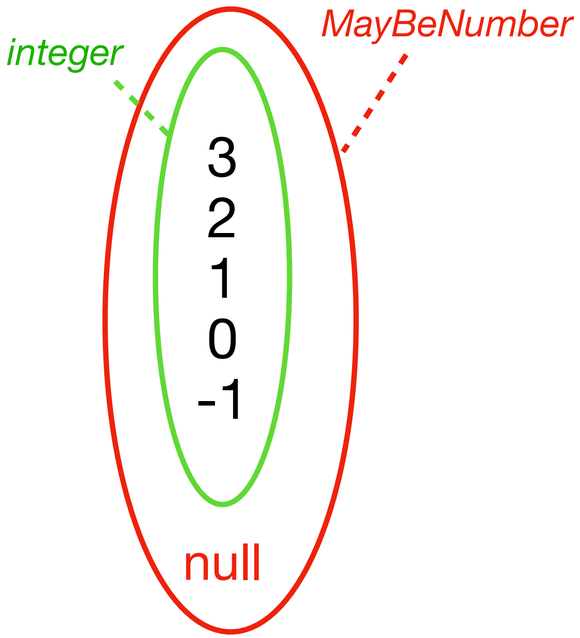
Null is a one billion dollar mistake.

By Tony Hoare who invented `null` in 1965 while designing the ALGOL W programming language.

Null Value

What's wrong with null?

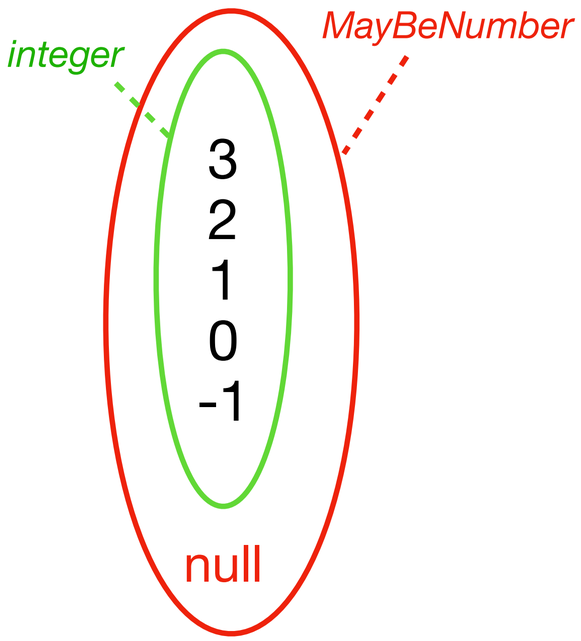
- Null doesn't have problem by itself.
- We are thinking about null value incorrectly.
- Think about the values and it's type, null isn't an element of it type!
- We should exclude null from output types unless we explicitly say they have the null value.
- Let's call number with nullable value -> MaybeNumber



จำนวนสมาชิกใน set

Cardinality

MaybeNumber has integer and null as members, this called cadinarity



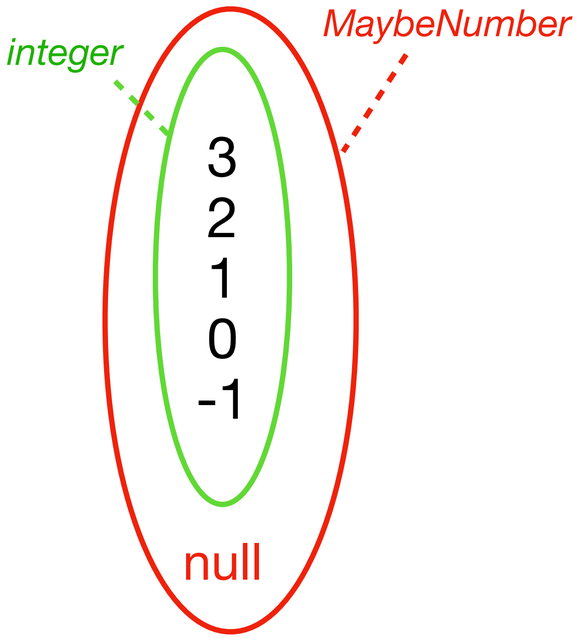
null, 1 means another one value

```
|MaybeNumber| = |number| + 1
```

- MaybeInteger is the **cardinality** of the Integer type plus 1
- **Cardinality** of a type, is a number of elements that type has.
- **Cardinality** of a type A represented as |A|

Maybe == Option Type in FP

Let's call Maybe as an Option type




```
type MaybeNumber = number | null  // <- what we have


type Option<A> = A | null         // <- define Optional of
                                     any type


type MaybeNumber = Option<number> // <- now, we can define a
                                     concrete type that reuse the
                                     generic Option type.
```

Some & None

Some and None are 2 variants type of the Option type

```
type Option<A> = Some<A> | None
```

```
interface Some<A> {
  readonly _tag: 'some'
  readonly value: A
}
```

```
const some = <A>(a: A): Option<A> => ({
  _tag: 'some',
  value:  a
})
```

```
interface None {
  readonly _tag: 'none'
}
```

```
const none: Option<never> = {
  _tag: 'none'
}
```

Partial Function

Let's go back and talk about compose Partial Functions

```
function divide2(x: number): number {
  return 2 / x
}

function increment(x: number): number {
  return x + 1
}

const chopBump = compose(increment, divide2)          // compile error

const chopBump = compose(
    x => isNone(x) ? none : some(increment(x.value)),  // chopBump(0) === none,
    divide2                                            // chopBump(8) === some(1.25)
)
```

There is another better way to solve this called Functor.
We'll talk about this later.

Lab : Understand Optional Type

1/3 - Define Option, Some, None

```ts
// option.ts

type Option<A> = Some<A> | None
type Some<A> = Readonly<{
  _tag: 'Some',
  value: A
}>
type None = Readonly<{
  _tag: 'None',
}>

export const some = <A>(value: A): Some<A> => ({
  _tag: 'Some',
  value
})

export const none: None = {
  _tag: 'None',
}

export const isSome = <A>(option: Option<A>): option is Some<A> =>
  option._tag === 'Some'

export const isNone = <A>(option: Option<A>): option is None =>
  option._tag === 'None'
```

```ts
// option.test.ts

import { isNone, isSome, none, some } from "./option"

describe('Option', () => {
  it('verify some option', () => {
    expect(isSome(some(1))).toBeTruthy()
    expect(isNone(some(1))).toBeFalsy()
  })

  it('verify none option', () => {
    expect(isNone(none)).toBeTruthy()
    expect(isSome(none)).toBeFalsy()
  })
})
```

Lab : Understand Optional Type

2/3 - Use Option Type

```
// option.compose.ts

import { pipe } from 'effect'
import { Option, some, none, isSome } from './option'

const divideTwo = (n: number): number => 2 / n
const liftUp = (n: number): Option<number> => n === 0 ? none : some(n)
const shouldDivideTwo = (o: Option<number>) => isSome(o) ? some(divideTwo(o.value)) : none

describe('Option Compose', () => {
  it('should compose two functions', () => {
    expect(
      pipe(
        1, liftUp, shouldDivideTwo
      )
    ).toEqual(some(2))
  })

  it('should return none if the input is 0', () => {
    expect(
      pipe(
        0, liftUp, shouldDivideTwo
      )
    ).toEqual(none)
  })
})
```

Lab : Effect Option

1/3 - Use Option Type Provided from Effect

```
// option.effect.test.ts

import { pipe } from 'effect'
import { Option, some, none, isSome } from 'effect/Option'

const divideTwo = (n: number): number => 2 / n
const liftUp = (n: number): Option<number> => n === 0 ? none() : some(n)
const shouldDivideTwo = (o: Option<number>) => isSome(o) ? some(divideTwo(o.value)) : none

describe('Option Effect', () => {
  it('should compose two functions', () => {
    expect(
      pipe(
        1, liftUp, shouldDivideTwo
      )
    ).toEqual(some(2))
  })

  it('should return none if the input is 0', () => {
    expect(
      pipe(
        0, liftUp, shouldDivideTwo
      )
    ).toEqual(none)
  })
})
```

Either

Preserve the error message that missing in Option

Either

can result either success or failure

- Option provide info that function may return some or none value.
- If you need to include additional information, such as an error message, to explain why a value is missing, Either is the preferred choice.

```
type Option<A> =    None | Some<A>

type Either<E, A> = Left<E> | Right<A>
```

Either

can result either success or failure

- The function should be Total Function. In the other hand, it should always “**Right**”.
- Otherwise, it should be “**Left**” off.

```
type Either<E, A>
```

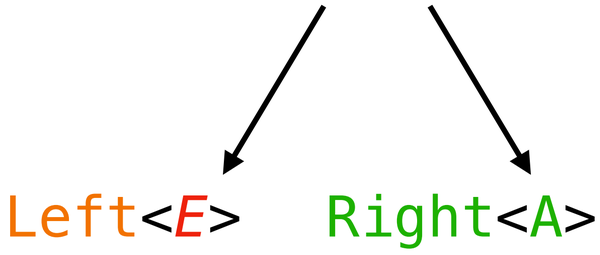
```
Left<E>   Right<A>
```

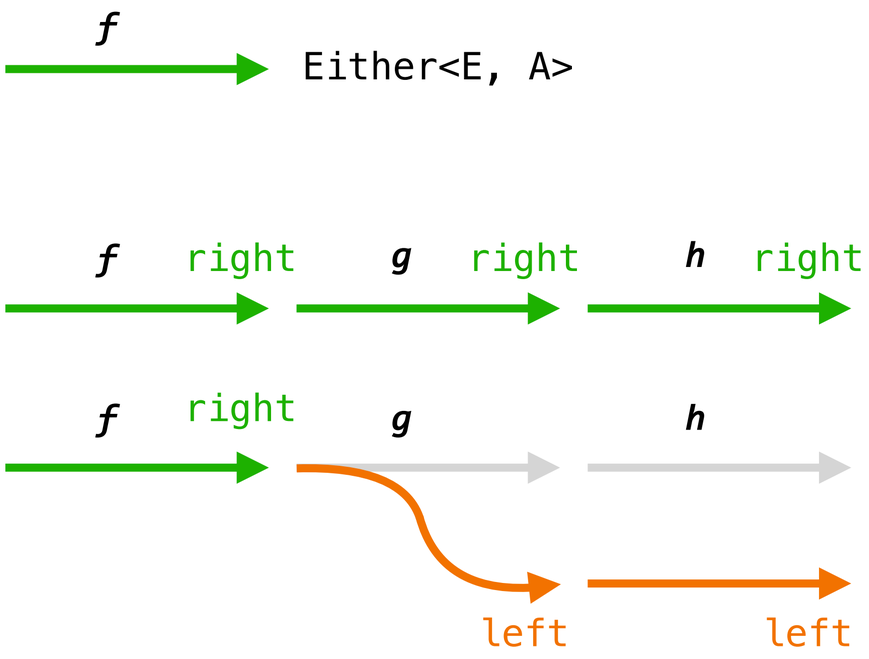
Either

Compose either functions

- Rather than handle error with try catch, fp use left or right result to handle error.

```
type Either<E, A>
```





Value or Error

this code is a Partial function we want to avoid

```
function DivideTwoIfEven(num: number): number {
  if (num === 0) {
    throw 'Cannot divide by zero'
  }
  else if (num % 2 == 0) {
    throw 'num is not even'
  }
  else {
    return 2 / num
  }
}
```

Option

Option not provide reason

```
function DivideTwoIfEven(num: number): Option<number> {
  if (num === 0) {
    return none
  }
  if (num % 2 == 0) {
    return none
  }
  return some(2 / num)
}
```

We want the type that can return error to provide the error reason.

Either Type

Type constructor

```
type Either<E, A> = Left<E> | Right<A>
```

Either Type

Implementation. Now, result can be left and right

```
type Either<E, A> = Left<E> | Right<A>

interface Left<E> {
  readonly _tag: 'left'
  readonly left: E
}
interface Right<A> {
  readonly _tag: 'right'
  readonly right: A
}

let result: Either<string, number>
result = { _tag: 'right', right: 12 }
result = { _tag: ‘left’,  left: 'Error' }
```

Either Type

Add helper function to left up any type to Either Left or Right

```
type Either<E, A> = Left<E> | Right<A>

interface Left<E> {
  readonly _tag: 'left'
  readonly left: E
}
interface Right<A> {
  readonly _tag: 'right'
  readonly right: A
}

let result: Either<string, number>
result = { _tag: 'right', right: 12 }
result = { _tag: ‘left’,  left: 'Error' }
```

```
const left = <E, A=never>(e: E): Either<E, A> => ({
    _tag: 'left', left: e
})

const right = <A, E=never>(a: A): Either<E, A> =>
({
    _tag: 'right', right: a
})
```

Either

Use Either to return none with error message

```
function DivideTwoIfEven(num: number): Either<string, number> {
  if (num === 0) {
    return left('Cannot divide by zero')
  }
  if (num % 2 !== 0) {
    return left('num is not even')
  }
  return right(2 / num)
}
```

Lab : Either

1/3 - Define Either, Left, Right

```
// either.ts

export type Either<E, A> = Left<E> | Right<A>

export type Left<E> = Readonly<{
  _tag: 'Left',
  value: E
}>

export type Right<A> = Readonly<{
  _tag: 'Right',
  value: A
}>

export const left = <E>(value: E): Left<E> => ({
  _tag: 'Left',
  value
})

export const right = <A>(value: A): Right<A> => ({
  _tag: 'Right',
  value
})

export const isLeft = <E, A>(either: Either<E, A>): either is Left<E> =>
  either._tag === 'Left'

export const isRight = <E, A>(either: Either<E, A>): either is Right<A> =>
  either._tag === 'Right'
```

Lab : Either

2/3 - Using Either

```
// divideTwo.test.ts

import { Either, left, right, isLeft, isRight } from "./either"

const divideTwo = (divisor: number): Either<string, number> => {
  if (divisor === 0) {
    return left('Division by zero error.')
  } else {
    return right(2 / divisor)
  }
}

describe('divideTwo', () => {
  it('should be Right when divisor is not zero, otherwise should be Left', () => {
    expect(isRight(divideTwo(4))).toBeTruthy
    expect(isLeft(divideTwo(0))).toBeTruthy
  })

  it('should return the result when divisor is not zero', () => {
    const result = divideTwo(4)
    if (isRight(result)) {
      expect(result.value).toBe(0.5)
    } else {
      fail('should be Right')
    }
  })
})
```

Lab : Either

3/3 - Using Either with Effect

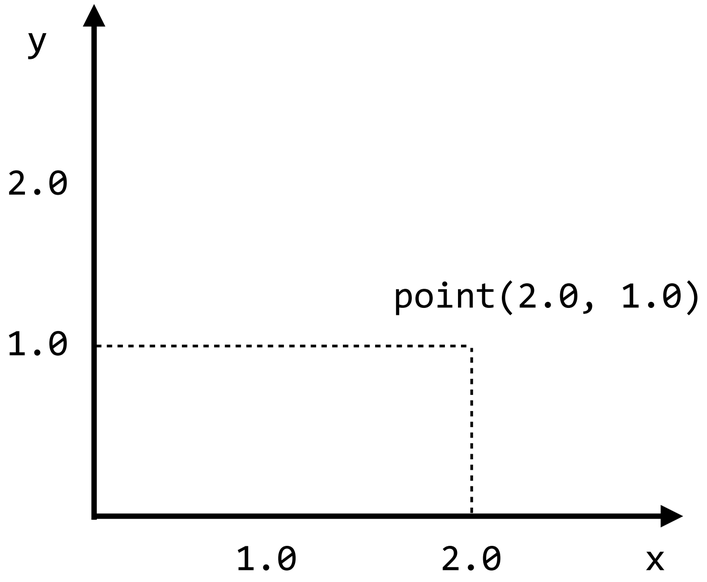
```
// divideTwo-effect.test.ts

import * as Either from "effect/Either"

const divideTwo = (divisor: number): Either.Either<number, string> =>
  divisor === 0
    ? Either.left("Division by zero error.")
    : Either.right(2 / divisor)

describe('divideTwo Effect', () => {
  it('should be Right when divisor is not zero, otherwise should be Left', () => {
    expect(
      divideTwo(4)
    ).toEqual(Either.right(0.5))

    expect(
      divideTwo(0)
    ).toEqual(Either.left("Division by zero error."))
  })

  it('should return the result when divisor is not zero', () => {
    const result = divideTwo(4)
    if (Either.isRight(result)) {
      expect(
        result.right
      ).toBe(0.5)
    } else {
      fail('should be Right')
    }
  })
})
```

ADT & Pattern Matching

Extract value with Pattern Matching

Compound Data Type

Group data objects into Tuples



Point in 2D plane

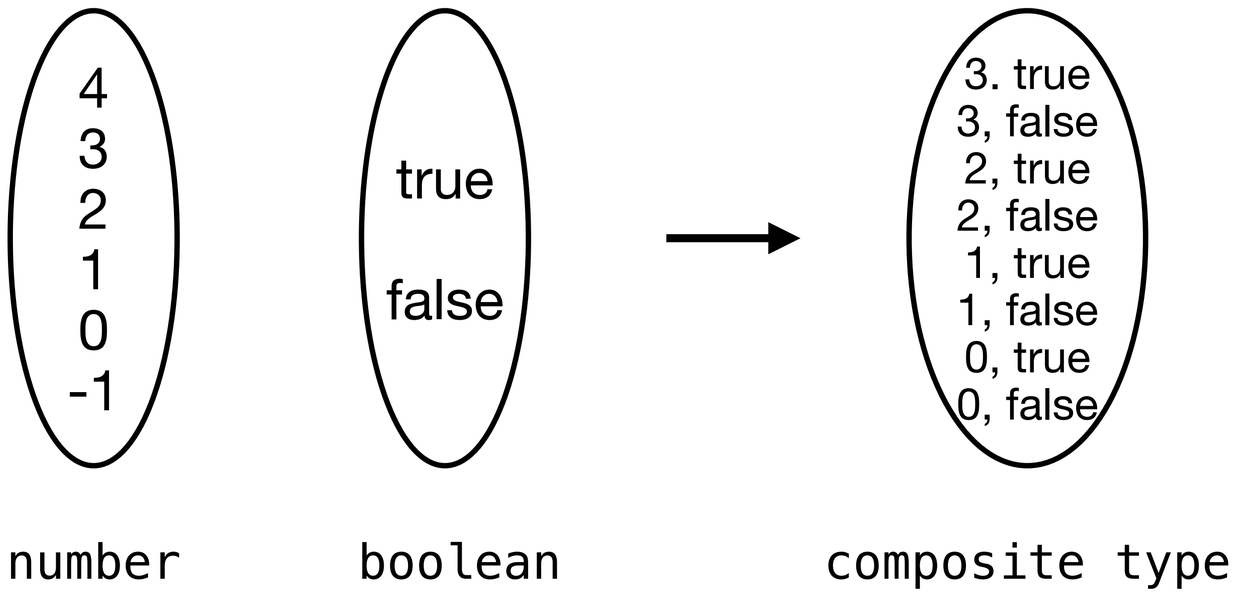
- Primitive data types are useful but often nor sufficient to modeling the real world phenomena.
- For example :- to represent the coordinates in 2D graphics. We need two numbers to represent x, y coordinator.
- We need to glue those x and y together into a compound data object as a single concept.
- In many programming languages support data type object to glue 2 or more data types together. It's called ***Tuple***.

```
const point: [number, number] = [1, 2]
const [x, y] = point
```

Composing Types

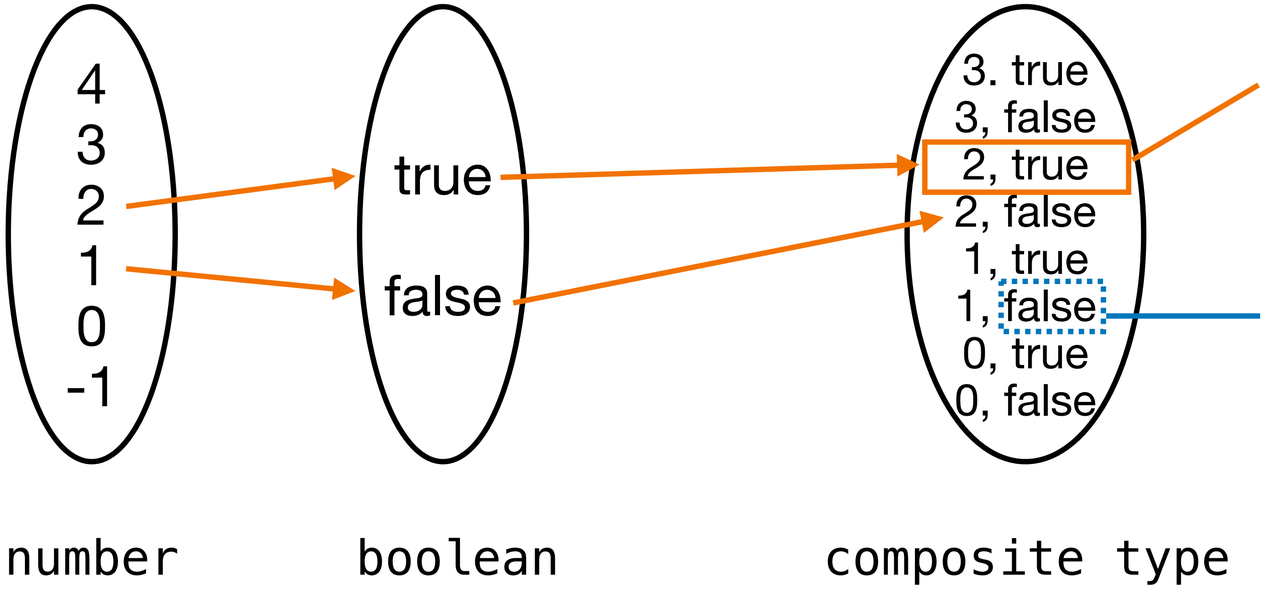
let's call it type

composing "set of values" with another "set of values" to create a "new set of values".



Composing Types

Product Type & Field



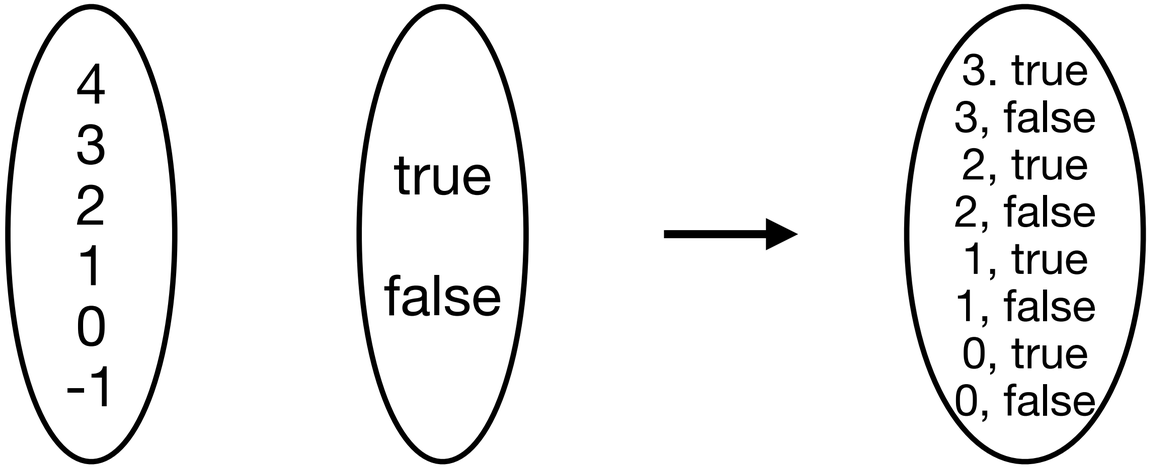
The resulting composite type is called Product Type.

In composite type, each value in side the pair is called a field in our new type.

Or called cardinality Type.

Composing Types

The values of composing type is the cardinality type of number x cardinality type of boolean



`number` `boolean`

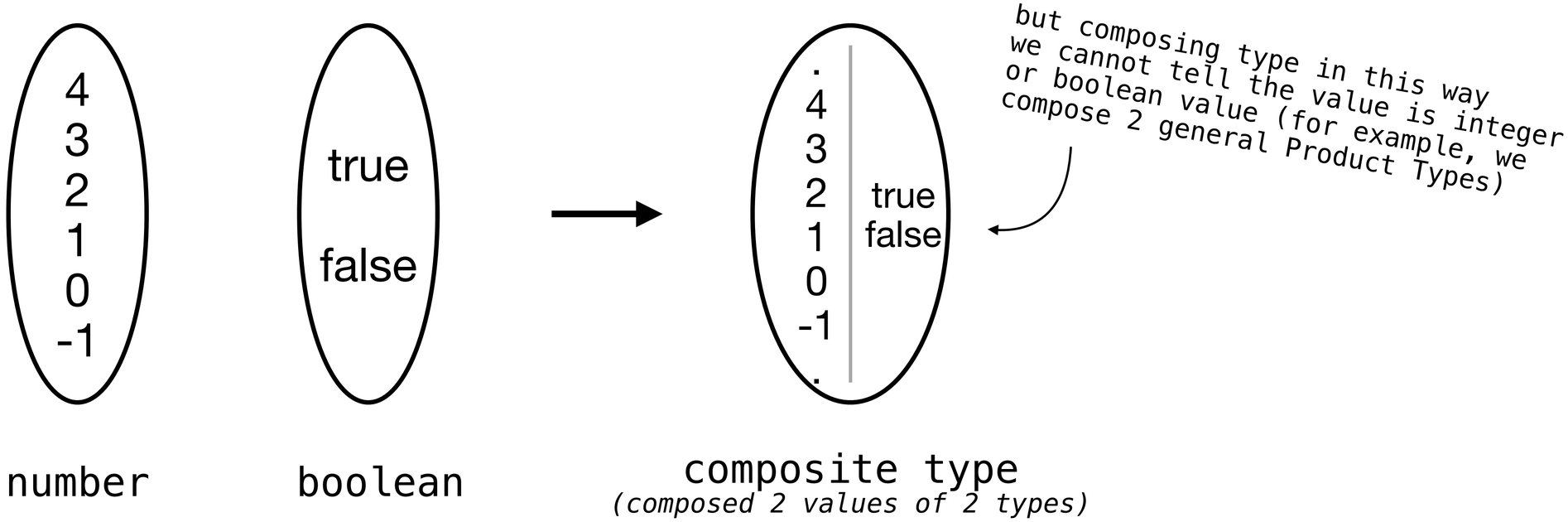
cardinality of number
x
cardinality of boolean

```
type UserRecord = {
  name: string
  age: number
  address: {
    city: string
    street: string
  }
}

type userTuple = [
  string,
  number,
  [
    string,
    string
  ]
]
```

Model Product Type in TypeScript

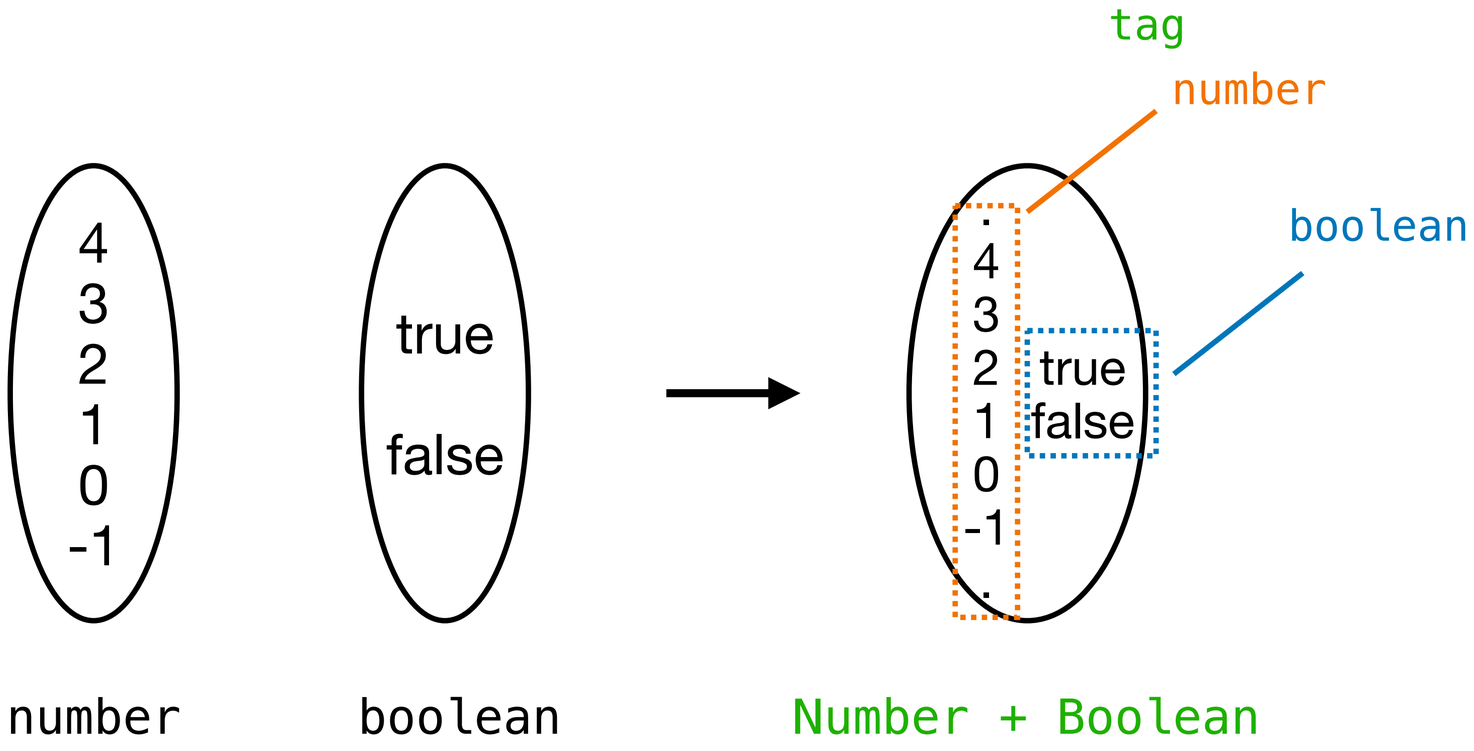
Composing Types

Putting every values from both type next to each other in our composite type.



Composing Types

Grouping and tagging the values in Composite Type



The composite type that compose like this called Sum Type.

ADT

- ADT stands for Algebraic Data Type.
- ADT is basically a composite type using operations.
  - Product operation
  - Sum operation
- We want to use composite type to store data inside them.
- The operation that work with composite type to extract data from it called *Pattern Matching*.

Algebraic Data Types

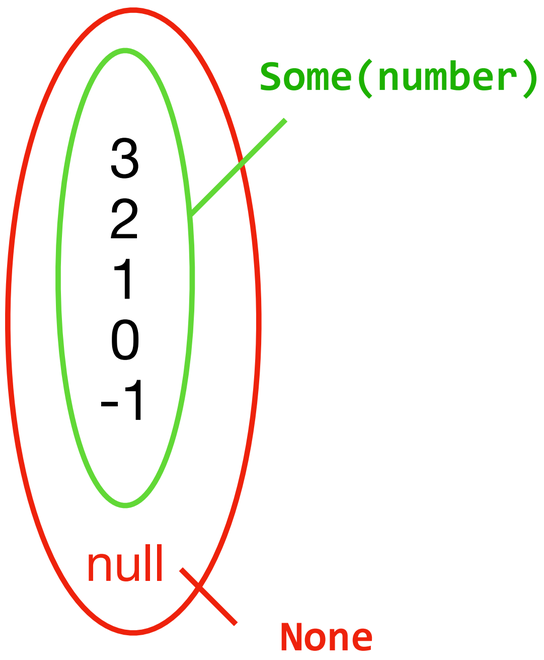
Example :-

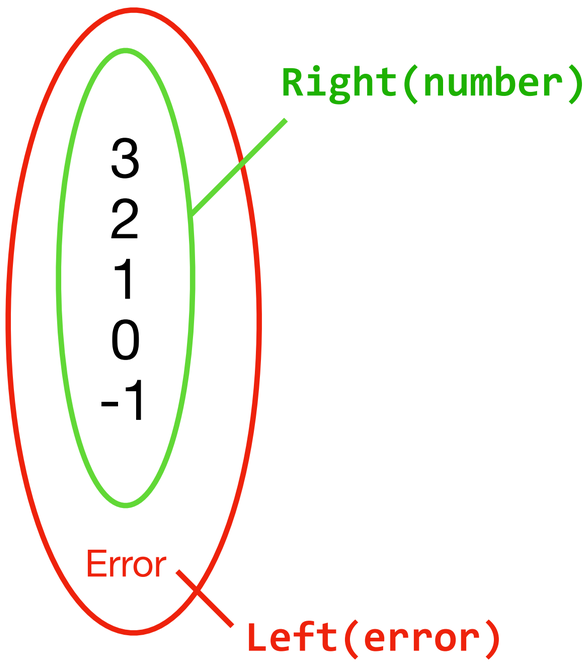
```
type DayOfWeek = 'Monday'
               | 'Tuesday'
               | 'Wednesday'
               | 'Thursday'
               | 'Friday'
               | 'Saturday'
               | 'Sunday'
```

```
type Shape = Circle
           | Square
           | Rectangle
           | Triangle

interface Circle {}
interface Square {}
interface Rectangle {}
interface Triangle {}
```

Option & Either

We've already seen ADT.






```
Option = Some<A> | None
```

```
Either = Left<E> | Right<A>
```

Model Sum Type in TypeScript

Unit Type or Either

sum of number and boolean

```
type NumberOrBool = number | boolean

const x: NumberOfBool = 5
const y: NumberOfBool = true
```

```
type Either<E, A> =
  | { _tag: 'left', left: E }
  | { _tag: 'right', right: A}
```

Either is a generic sum type that can have a left object type or right object type.

```
const x: Either<string, number[]> = { _tag: `left`, left: `Error` }
const y: Either<string, number[]> = { _tag: `right`, left: [1, 2, 3] }
```

Convert composition type to other type

Let's see option type

2 variants, none & some<boolean>

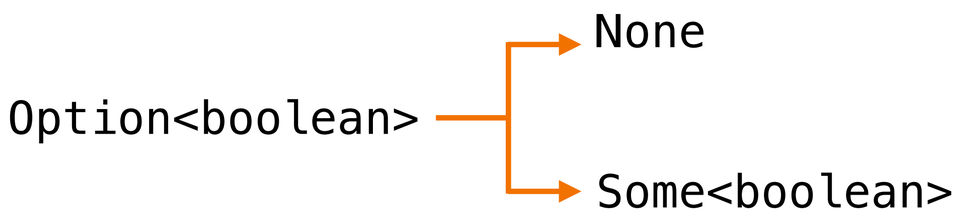
```
type Option<boolean> = None | Some<boolean>
```

we want to extract value from 2 variants

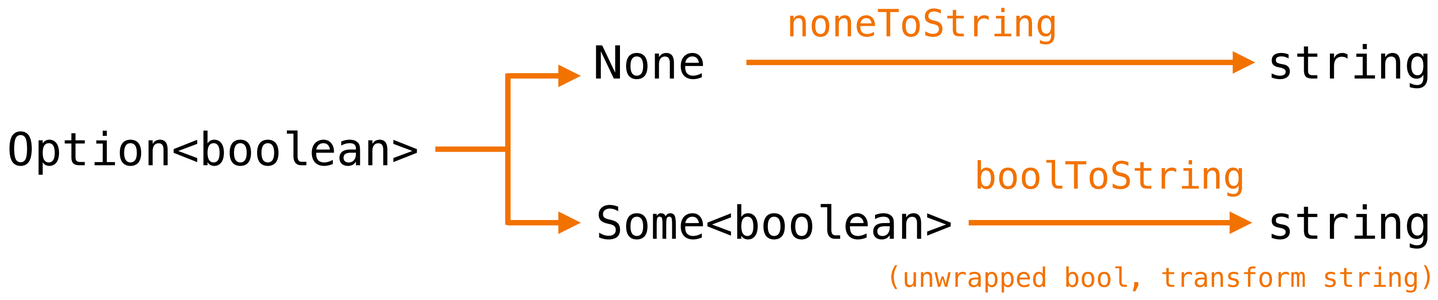
```
Option<boolean> --toString--> string
```

Pattern Matching

Extract and transform, from Option<boolean> to string.



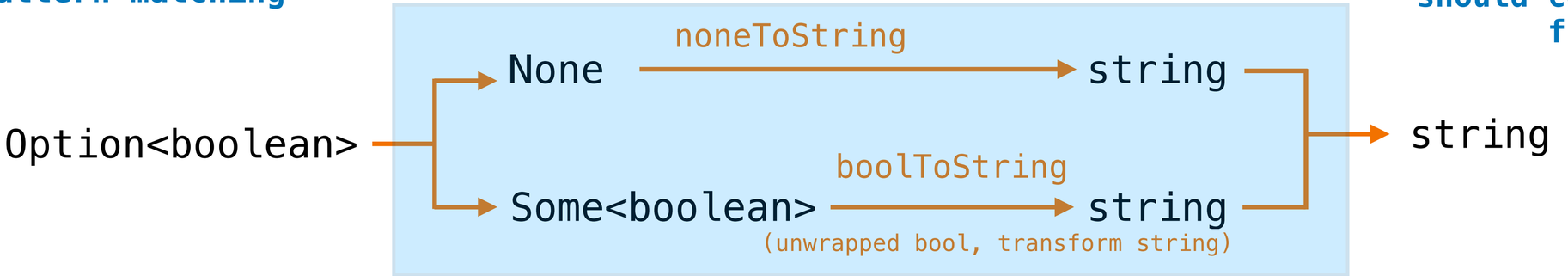
we look at each variant in option (composite type) case by case then extract data from each variant and transform all of the result to a desired value type.



Because when implement pattern matching un FP, FP don't have partial function.

That's means, Pattern matching should cover all the values from its input type.

This called, pattern matching



Lab 1 : Pattern Matching - Option

1/3 - Option

```
// Option ------------------------

type Option<A> =
  | None
  | Some<A>

type None = Readonly<{
  _tag: 'none'
}>

type Some<A> = Readonly<{
  _tag: 'some'
  value: A
}>

const some = <A>(x: A): Option<A> => ({ _tag: 'some', value: x })
const none: Option<never> = { _tag: 'none' }
const isNone = <A>(x: Option<A>): x is None => x._tag === 'none'

export { Option, some, none }
```

```
interface None {
  readonly _tag: 'none'
}

interface Some<A> {
  readonly _tag: 'some'
  readonly value: A
}
```

Lab 1 : Option

2/3 - Option Type Pattern Matching

```
// Pattern Matching -------------------------

type Match = <A, B>(
  onNone: () => B,
  onSome: (a: A) => B
) =>
  (x: Option<A>) => B

const match: Match = (onNone, onSome) => x =>
  isNone(x) ? onNone() : onSome(x.value);

export { Option, some, none, match }
```

```
// Test------------------

import { match, Option, some } from "./adt-option"

describe('Pattern Matching with Option Type', () => {
  it('should match Option type to string', () => {

    const mayBeNum: Option<number> = some(10)
    const matched = match(
      () => `number doesn't exist`,
      (x) => `${x}`
    )(mayBeNum)

    expect(matched).toEqual('10')

  })
})
```

Lab 1 : Option

3/3 - MatchW (Widen)

```
...
const match: Match = (onNone, onSome) => x =>
  isNone(x) ? onNone() : onSome(x.value)

type MatchW = <A, B, C>(
  onNone: () => B,
  onSome: (a: A) => C
) =>
  (x: Option<A>) => B | C

const matchW: MatchW = (onNone, onSome) => x =>
  isNone(x) ? onNone() : onSome(x.value)

export { Option, some, none, match, matchW }
```

```
it('should match widen Option type to number when it is some type', () => {
  const mayBeNum: Option<number> = some(10)
  const matched = matchW(
    () => `number doesn't exist`,
    (x) => x
  )(mayBeNum)
  expect(matched).toEqual(10)
})
```

```
it('should match widen Option type to string when it is none type', () => {
  const mayBeNum: Option<number> = none
  const matched = matchW(
    () => `number doesn't exist`,
    (x) => x
  )(mayBeNum)
  expect(matched).toEqual(`number doesn't exist`)
})
```

Lab 2 : Pattern Matching - Either

1/2 - Either

```
// Either -------------------------

type Either<E, A> = Left<E> | Right<A>

type Left<E> = Readonly<{
  _tag: 'left'
  left: E
}>

type Right<A> = Readonly<{
  _tag: 'right'
  right: A
}>

const left = <E, A = never>(e: E): Either<E, A> => ({
  _tag: 'left', left: e
})

const right = <A, E = never > (a: A): Either<E, A> => ({
  _tag: 'right', right: a
})

const isLeft = <E, A>(x: Either<E, A>): x is Left<E> =>
  x._tag === 'left'

export { Either, left, right, isLeft }
```

Lab 2 : Pattern Matching - Either

2/2 - Pattern Matching Either

```
// Pattern Matching -------------------------

type Match = <E, A, B>(onLeft: (e: E) => B, onRight: (a: A) => B) =>
  (x: Either<E, A>) => B

const match: Match = (onLeft, onRight) => x =>
  isLeft(x) ? onLeft(x.left) : onRight(x.right)

export { Either, left, right, isLeft, match }
```

```
it('should match right to string', () => {
  const errorOrNum: Either<string, number> = right(12)
  expect
    (match(
      (e: string) => e,
      (a: number) => `value is ${a}`
    )(errorOrNum)
  ).toEqual(`value is 12`)
})
```

```
it('should also match left to string', () => {
  const errorOrNum: Either<string, number> = left(`error`)
  expect(
    match(
      (e: string) => e,
      (a: number) => `value is ${a}`
    )(errorOrNum)
  ).toEqual(`error`)
})
```

[x] Lab 3 : Pattern Matching - List

1/3 - List (optional lab)

```
// List ----------------------------------------

type List<A> = Nil | Cons<A>

type Nil = Readonly<{
  _tag: 'nil'
}>

type Cons<A> = Readonly<{
  _tag: 'cons'
  head: A
  tail: List<A>
}>

const nil: List<never> = { _tag: 'nil' }
const cons = <A>(head: A, tail: List<A>): List<A> => ({
  _tag: ‘cons', head, tail
})
```

```
const isNil = <A>(xs: List<A>): xs is Nil => xs._tag === 'nil'

const showList = <A>(xs: List<A>): string =>
  isNil(xs)
    ? ''
    : `${xs.head}` +
      `${isNil(xs.tail) ? "" : ", "}` +
      `${showList(xs.tail)}`


// Pattern Matching ----------------------------------------

// Start hete…



export { Cons, cons, nil, showList }
```

[x] Lab 3 : Pattern Matching - List

2/3 - List Pattern Matching

```
// Pattern Matching -------------------------------

type Match = <A, B>(
  onNil: () => B,
  onCons: (xs: List<A>) => B
) => (xs: List<A>) => B

const match: Match = (onNil, onCons) => xs =>
  isNil(xs) ? onNil() : onCons(xs)

export { Cons, cons, nil, showList, match }
```

```
import {
  Cons, cons, match, nil, showList
} from ‘./adt-list'

describe('Pattern Matching with Option Type', () => {
  it('should match list to string', () => {
    const list = cons(1, cons(2, cons(3, cons(4, nil))))
    expect(
      match(
        () => '',
        (xs) => showList(xs)
      )(list)
    ).toEqual('1, 2, 3, 4')
  })

  it('should match null list to empty string', () => {
    expect(
      match(
        () => '',
        (xs) => showList(xs)
      )(nil)
    ).toEqual('')
  })
})
```

[x] Lab 3 : Pattern Matching - List

3/3 - List Pattern Matching with ts-pattern

```
import {
  Cons, cons, match, nil, showList
} from './adt-list'

import { match as matchFp} from 'ts-pattern'

describe('Pattern Matching with Option Type', () => {

// ...

  it('should match list to string with ts-pattern', () => {
    const list = cons(1, cons(2, cons(3, cons(4, nil))))
    expect(
      matchFp(list)
        .with({ _tag: 'nil' }, () => '')
        .with({ _tag: 'cons' }, (xs) => showList(xs))
        .exhaustive()
    ).toEqual('1, 2, 3, 4')
  })
})
```

Kind,
Higher-Kinded Types

Type that accept another type and return a new Type

Let’s Recap

from earlier lesson

- We learn that values grouped by types.
- We know the benefit about coding by dealing with types rather than values.
- Functions are simply mapping between different types.
- Higher-order function that can pass function and return function.
- Next, we will learn about another abstraction above types and type constructors.

The Value and Type Universe

start by looking at values again, then type, and something above of its.

Types made compiler help us
to prevent error
when dealing with different values.

Value Universe	Type Universe	???
12	number	There are another universe that abstract the similar patterns in the Type Universe.
`hello`	string	
true	number => boolean	
	Option<boolean>	

Option

Think about Option

- **Option** itself seem to be incomplete. We need to pass a concrete type to it's type constructor.

```
Option<???>  ->  Option<boolean>

Option: Type ->  Type
```

boolean

Option<boolean>

- It's acting like functions that accept another type (boolean) to construct itself (option). And this is how we abstract over the Type universe.

- We called the type, like Option, that abstract type in to the higher level as ***Kind***.

Kind

The abstraction level that abstract the Type Universe into the higher level.

Value Universe

```
12

`hello`

true
```

Type Universe

```
number

string

number => boolean

Option<boolean>

string[]
```

Kind Universe

```
Type -> Type

A[]

Option<A>
  -> some<A>
  -> none

(Type -> Type) -> Type

Either<E,A>
  -> Left<E>
  -> Right<A>
```

Kind

the definition

- Kind universe is an ***abstraction*** over the Type universe.
- Kind is a classification for *types* and *type constructors*.
- Type constructors in the Kind universe are like *currying functions* in the Type universe.

```
Either<E, A>    =>     Either -> E -> A
```

- Think about array for example. Array keeps a sequence of items with a same type. It's doesn't care about the items it holds. All the operations are what array care about.

```
Array<Number>    [1, 2, 3, 4]
Array<boolean>   [true, false, true]
Array<string>    [`hello`, `world`, `!`]
```

- In the other word, Array is *abstracting over the type* it holds.

Higher-Kinded Type

look at the Kind closely…

- Looking at the Array again…

```
Array<Number>     [1, 2, 3, 4]
Array<boolean>    [true, false, true]
Array<string>     [`hello`, `world`, `!`]

Array<A>          A is any type
```

- Then, we also have another Kind universe…

```
Array <Number>    [1, 2, 3]
List  <Number>    cons(1, cons(2, cons(3, nil)))
Option<Number>    some(1)
```

- Can we have a generic type constructor that represents the common behaviour between type constructors?

- **The answer is YES. That's called *Higher-Kinded Type*.**

F<Type>

Higher-Kinded Type

A	Type (like Number)
Array<A>	Life up A with Array of A **Type -> Type**
F<A>	F can be Array, List, Option of A. (F is a Higher-kinded Type)
Array<A>	Replace F with Array **Type -> Type**
Array<Number>	**(Type -> Type) -> Type**

Higher-Kinded Type

Each Functor has map function, but the implementation is different

Type -> Type

```
type map_option =      <A, B>(f: (a: A) => B) => (x: Option<A>)     => Option<B>
type map_List   =      <A, B>(f: (a: A) => B) => (x: List<A>)       => List<B>

type map_either =   <E, A, B>(f: (a: A) => B) => (x: Either<E, A>) => Either<E, B>
```

Type -> Type -> Type

Type Class

To simplify it in to Type -> Type

Type -> Type

```
type map_option =      <A, B>(f: (a: A) => B) => (x: Option<A>)    => Option<B>
type map_List   =      <A, B>(f: (a: A) => B) => (x: List<A>)      => List<B>

type map_either =   <E, A, B>(f: (a: A) => B) => (x: Either<E, A>) => Either<E, B>

type map_either<E> =   <A, B>(f: (a: A) => B) => (x: Either<E, A>) => Either<E, B>

Functor<F> {    <- This called Type Class
    map: <A, B>(f: A => B) => F<A> => F<B>
}
```

We'll talk about Type-Class in the next chapter.

Type Class

The concept behind Function Composition

Type Class

Concept - From mathematical reasoning

- ***Type class*** is essentially a sort of interface that defines a set of functions or behaviors that a **type** must implement.
- It's a way of saying, "Any type that wants to be in this club needs to provide these particular methods." In short, type classes are about defining shared behavior.
- Type classes start small, abstraction, **defining simple rules** for certain concepts. And through **mathematical reasoning**, they defined larger, more complex concepts that derive from and depends on other concepts. These concepts applied to many domains.
- As long as the rules are followed the computations are **guaranteed to produce correct results** without going into immediate specifics of a given domain.
- To be able to program in functional style, we first need to **define these abstract concepts**, and then learn **how they relate to one another**, and how they **can be composed** to achieve the **desired computation results**.

Polymorphism

Let's recap about polymorphism

- Polymorphism consists of 2 words, poly and morphism.
- Polymorphism is the ability of an entity to take multiple shapes or forms.
- Example in OOP :-

```
interface Equal {
    isEqual(to: Equal): boolean
}

class Customer implements Equal {
    public name: string = ""
    public lastName: string = ""

    isEqual(to: Equal): boolean {
        let c = to as Customer
        return this.name === c.name &&
               this.lastName === c.lastName
  }
}
```

```
function compare(obj1: Equal, obj2: Equal): boolean {
    return obj1.isEqual(obj2)
}
```

Parametric Polymorphism

or Generic

- **Parametric Polymorphism** also known as ***Generic***.
- Allow for definition of functions without specifying an exact type.
- Example :-

```
function toString<A>(a: A): string {
  return `${a}`
}

toString(12)
toString(`hello`)
toString(true)
```

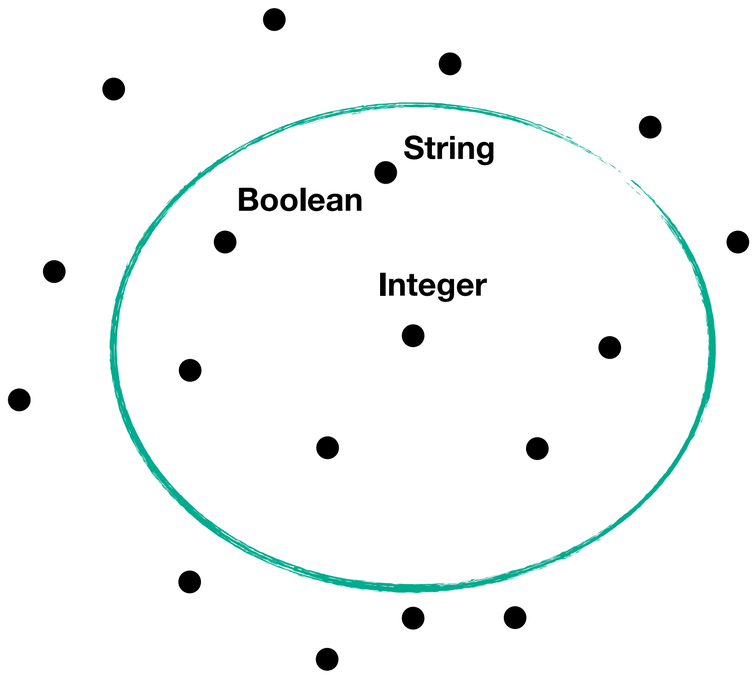
Ad-hoc Polymorphism

used wildly in TypeScript Function Programming.

- Ad-hoc Polymorphism specifies different behaviour ***based on the input type***.
- In TypeScript, we use interface type with constant of function.
- Since implementation is a constant. We can pass its to function or return it from function. This lead to a way of organize code know as the ***Strategy Pattern***.
- **Type class implement polymorphism in this way.**

```
type Show<A> = {
  toString: (a: A) => string
}

const showString = <A>(s: Show<A>) => (a: A) => s.toString(a)  // <- show is a
                                                                  higher-order function

const fromNumber:  Show<number> =  { toString: (a) => a.toString() }
const fromString:  Show<string> =  { toString: (a) => a }
const fromBoolean: Show<boolean> = { toString: (a) => a.toString() }
```

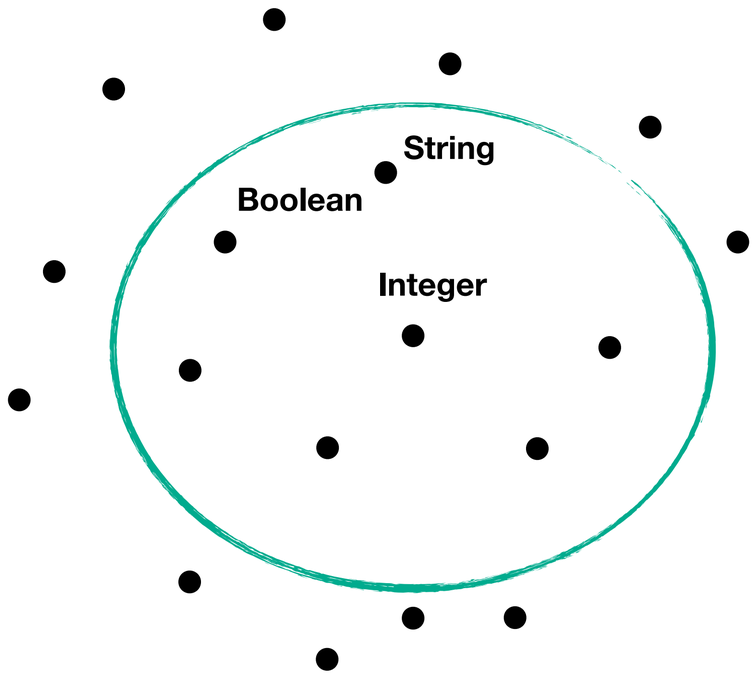
From Some Types



- Let's think about types are points in the space.
- Then, we have a function that we want to convert values of any type to string.

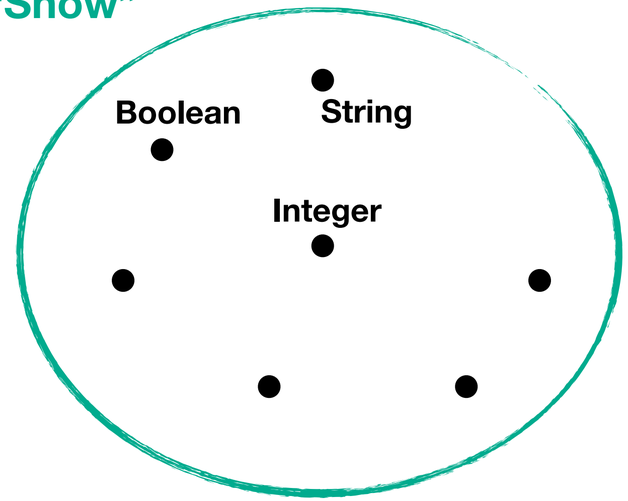
Types

and group of Types



- Let's draw a **line** cover all the types that we "***can convert to string***".

let's call it "Show"

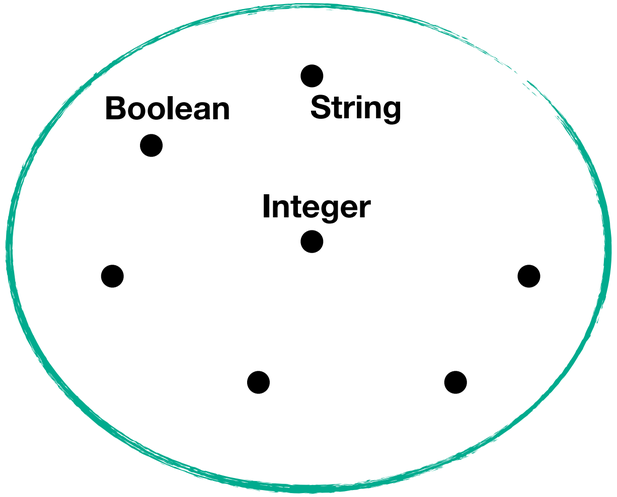


- And group those types and call it **"Show"**

Types

Looks what in common…

Show as a Kind Type

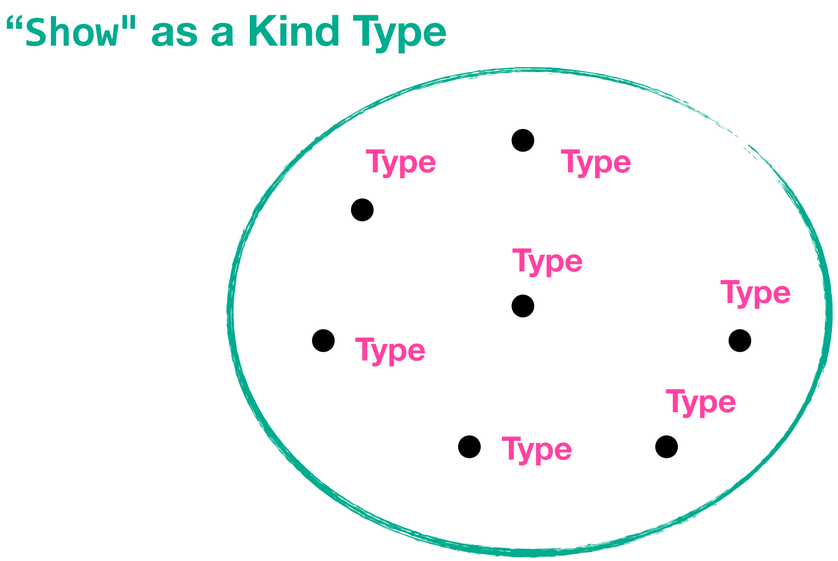


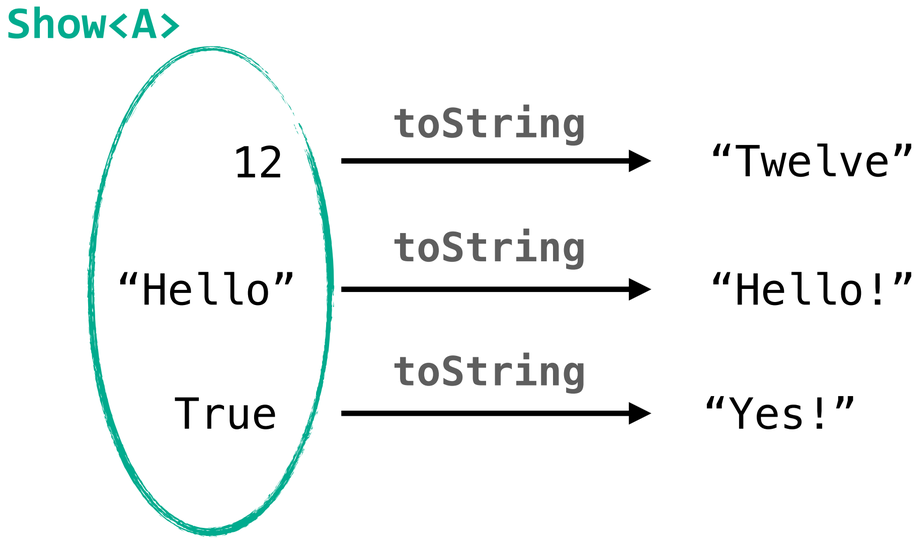

```
12        --toString-->  "Twelve"
"Hello"   --toString-->  "Hello!"
True      --toString-->  "Yes!"
```

- In this context, we're not focus on what a string values from toString function.
- We know that the actual process to convert to string is different and its depends on what is the input type.
- But right now we're only focusing on the possibilities of the action.
- This gives us the idea that may be we can **abstract** the toString functions into something **more generic at a higher level**.

Types

Looks what in common…





- So, rather than looking at these as an individual type. We can see it as T**ype** in the **Kind** Universe.
- Then we can simplify the mapping into **one polymorphic** toString that work with any type that work with our **"Show"** circle.

Types

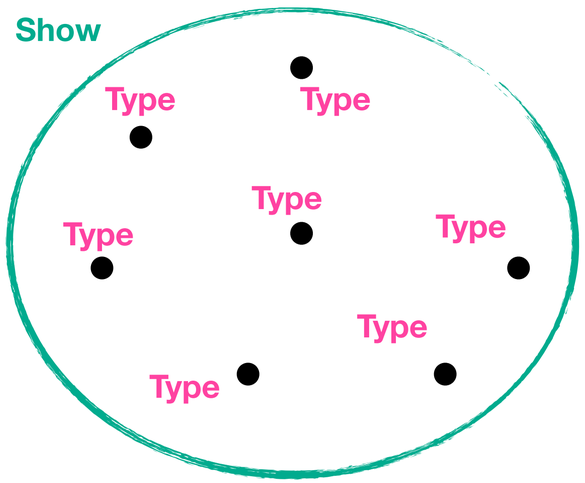
Looks what in common…

“Show" as a Kind Type

- Since everything inside the Show circle looks the same, we can now thing of show as a single entity and simplify out definition of the toString.
- Now, the toString function simply receives ***Show<A>*** and returns a ***String***.
- We call show “TypeClass".

Type Class

*The tool in Functional Languages that let's us **group** and **classify** types based on behaviour they exhibit is called **Type Class.***



Example in TypeScript

$$\text{Show} \xrightarrow{\texttt{toString}} \text{String}$$

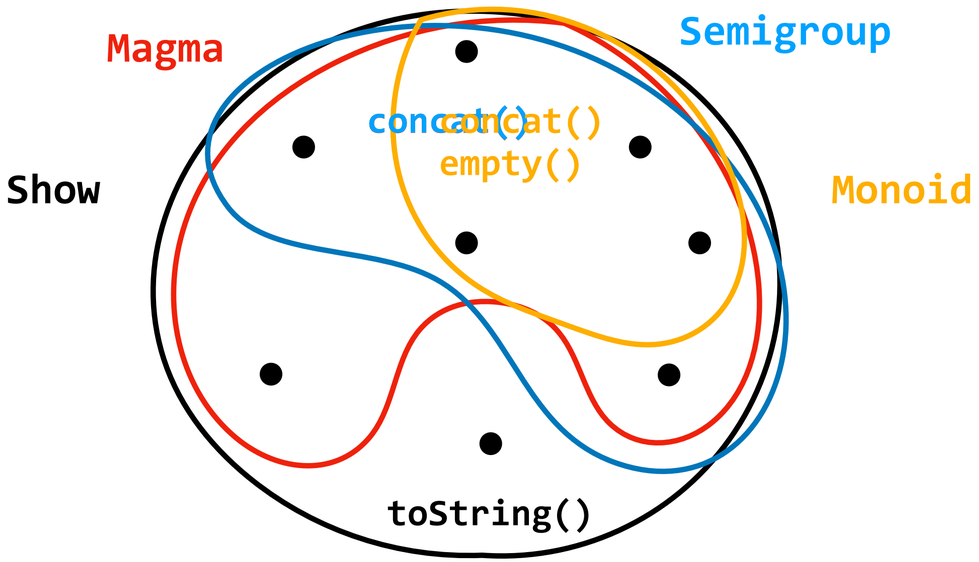
let's see TypeScript example again

```
export type Show<A> = {
  f: (a: A) => string
}

// Show -> A -> string
export const toString = <A>(s: Show<A>) => (a: A) => s.f(a)

export const num:  Show<number>  = { f: a => `${a}` }
export const str:  Show<string>  = { f: a => a }
export const bool: Show<boolean> = { f: a => a ? 'true' : 'false' }

describe('toString', () => {
  it('should return the string representation of any type to string', () => {
    expect(toString(num)(1)).toEqual('1')
    expect(toString(str)('hello')).toEqual('hello')
    expect(toString(bool)(true)).toEqual('true')
    expect(toString(bool)(false)).toEqual('false')
  })
})
```

Type Class

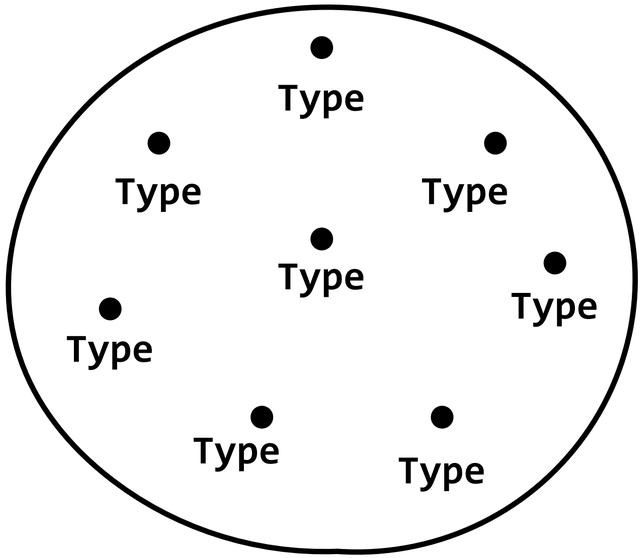
some of it, and more

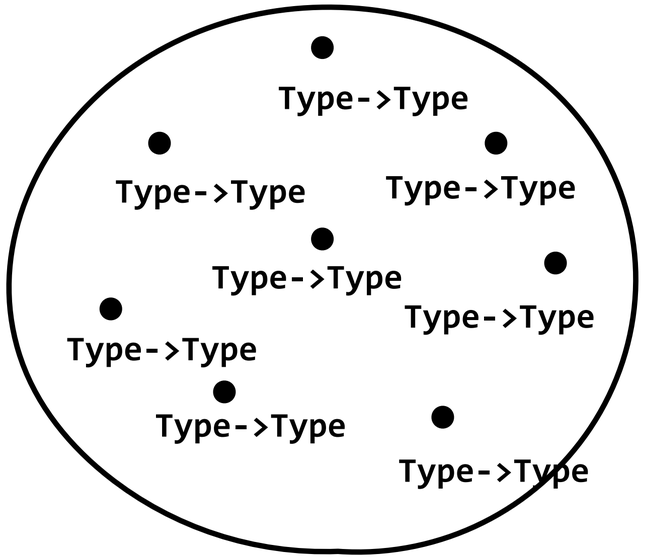
```
interface Show<A> {
  toString: (a: A) => string
}

interface Magma<A> {
  concat: (a: A, b: A) => A
}

interface Monoid<A> extends Semigroup<A> {
  concat: (a: A, b: A) => A
  empty: => A
}

interface Semigroup<A> extends Magma<A> {
  concat: (a: A, b: A) => A
}
```



As long as TypeScript not support Type Class natively, we simply use Interface class to implement the idea of the Type class. However, Interface class and Type class is came from different perspectives.

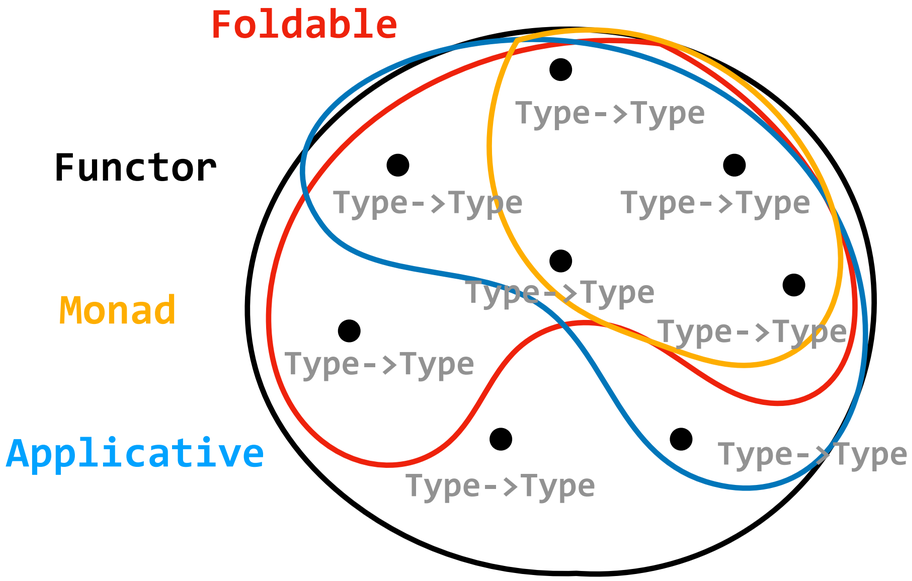
Type Class of Kind (Type -> Type)






```
Tree
List
Array
Option
Either<E,
...
```

Type Class of Kind (Type -> Type)



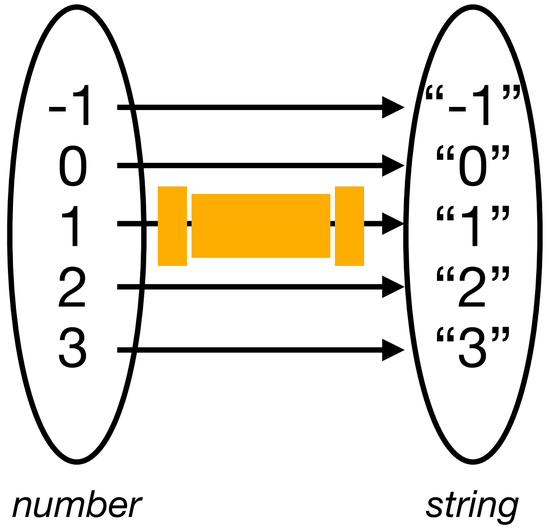

```
Functor f => (a -> b) -> f a -> f b

Applicative f => f (a -> b) -> f a -> f b

Monad m => m a -> (a -> m b) -> m b

Foldable t => (a -> b -> b) -> b -> t a -> b

    example f a -> f b :-

      Array<a>      -> Array<b>
      List[a]       -> List[b]
      Option<a>     -> Option<b>
      Either<e, a> -> Either<e, b>
```

IO

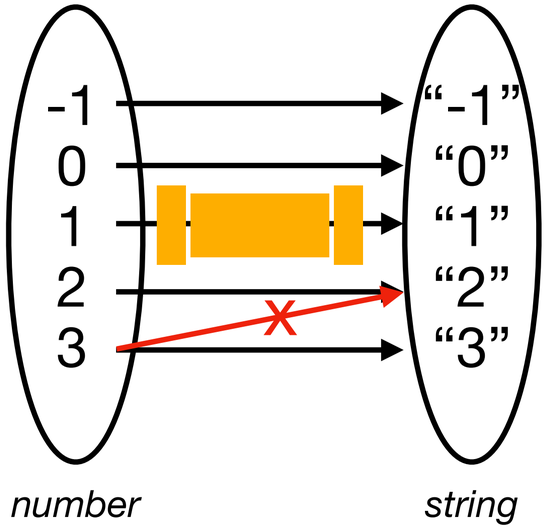
Compose synchronous computations with Side Effects

Pure Functions

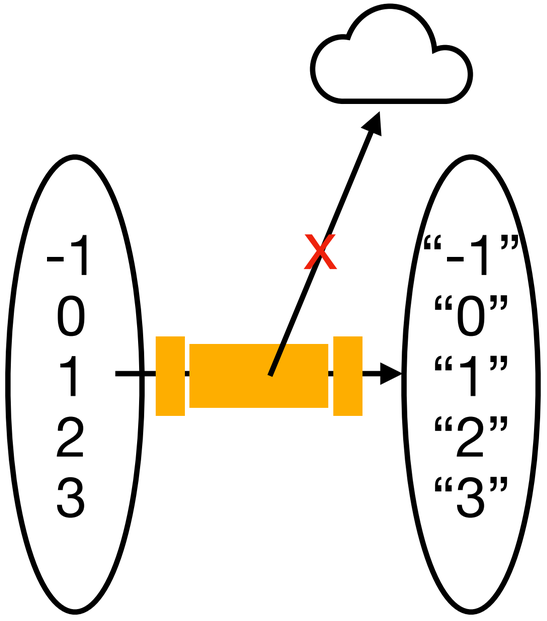
revisit function definition



Total Function



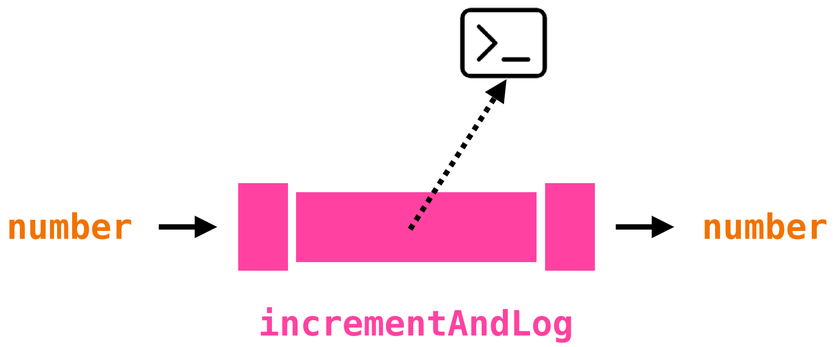
Deterministic



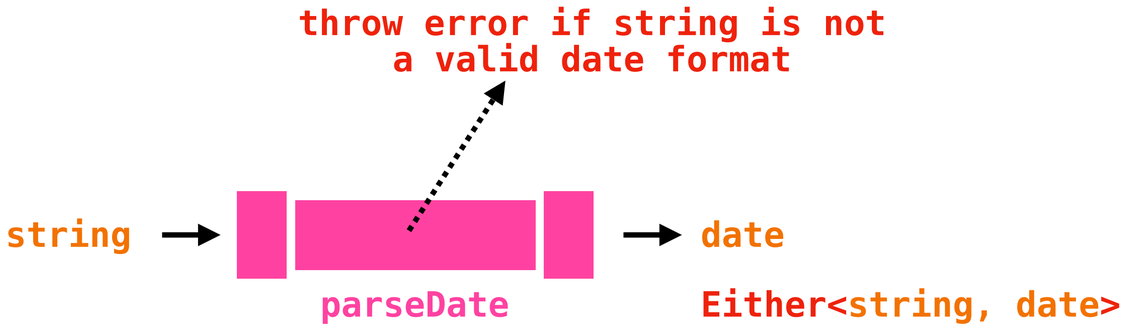
No Side effects

Side Effects

it implicitly dependent on the context



write a message to console that may not be able to compose



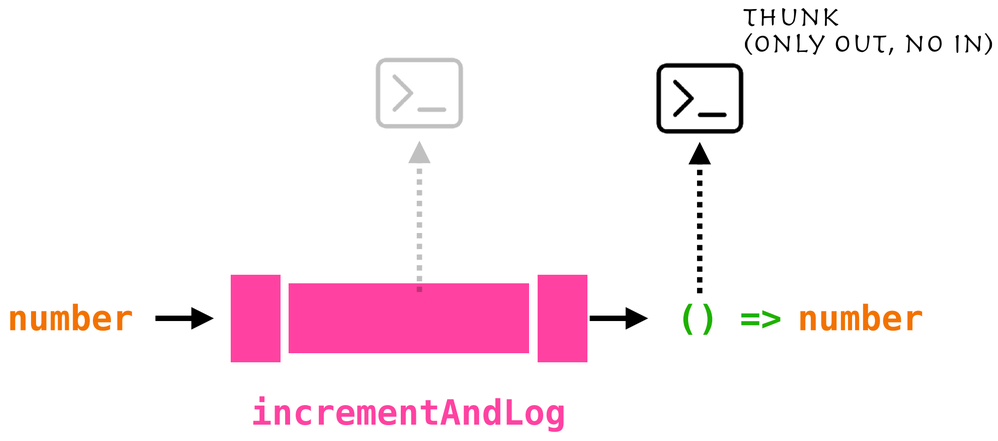
however, think about the error that happened when compose functions.

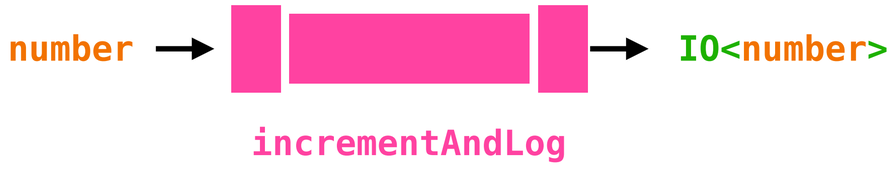
We've already solved this problem with Either.

We'll use this idea to model any kinds of Side Effects

Thunk

a pure function that has no input, handle side effect, and always return the same result.





- We can add small function that accept no argument that extract the side effect part into a new small function.
- We called this function “**Thunk**”.
- This make “`incrementAndLog`” pure.
- To modeling side effects, we came up with a new structure specifically to handle side effect. Let’s named it - IO.

IO

```
type IO<A> = () => A
```

- **IO** model computational

  IO wrap generic type constructor over any type ‘A’. It’s a computational wrapped in a box so that we can pass it around as data. And return value based on type it wrapped. This function doesn’t receive and input and always return a value of type ‘A’.

- **IO** has side effects

  It can get or set values outside of its body.

- **IO** has no exception

  When execute IO. It’s doesn’t error out.

- **IO** is synchronous

  It should be synchronous. When we execute the IO, it will block the execution until it’s done.

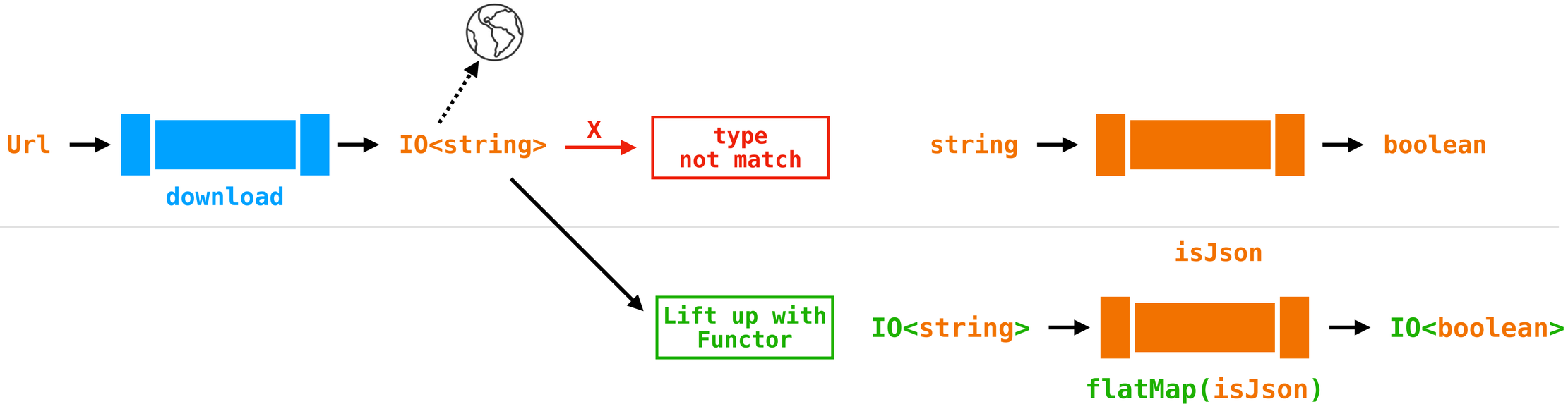
IO

example

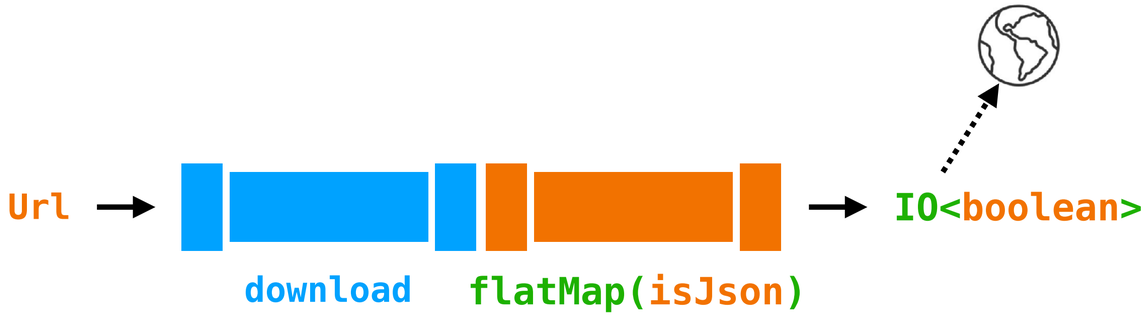
```
const increment = (value: number): number => {
  const output = value + 1
  console.log(output)
  return output
}

increment(5)

// ==========================================================

type IO<A> = () => A

const ioIncrement = (value: number): IO<number> => {
  const output = value + 1
  return () => {
    console.log(output)
    return output
  }
}

const incrementAndLog = ioIncrement(5)
incrementAndLog()
```

Composing IO

Lift up with functor to map string (input) to IO context

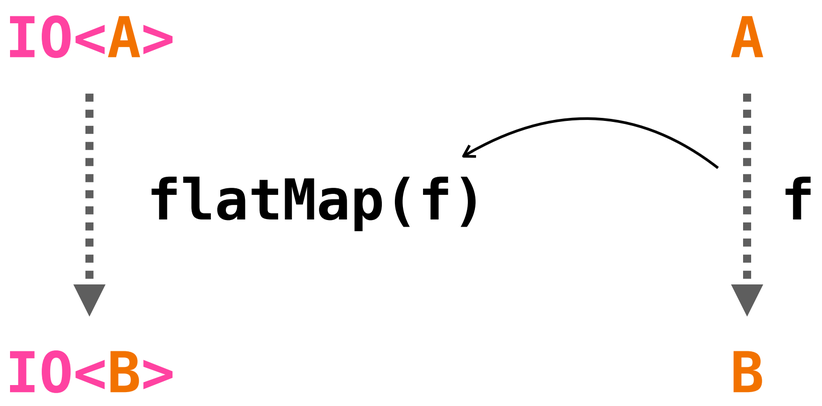


Now we can compose 2 pure/ reliable functions together and delay side effect handling to the edge of the application.



IO as a Functor

IO that can be map from type to type



IO with Error

Handling error with Either - IOEither

```
IO<Either<E, A>>        IOEither<A>
        ⋮                    ⋮
   flatMap(f)           flatMap(f)
        ▼                    ▼
IO<Either<E, B>>        IOEither<B>
```

Since IO and Either are Functor, we can compose then into a new Functor.

IO Summary

IO is not a function - it's a structure that holds data

- **IO** Provide us a tool to handle ***side effects***.
- **IO** is a covariant ***Functor***.
- **IO** does not error out and always provide its wrapped type.
- **IO<Either<E, A>** can be composed to create **IOEither<E, A>**.
- **IO** delays running ***Synchronous*** side effect to a later time.
- **IO<Promise<A>>** can be used to model ***Asynchronous*** side effect.
- **IO<Promise<A>>** can be composed to create **Task<A>**.

Lab 1 : IO

```
import { Effect, Console } from "effect"
import { pipe } from "effect/Function"

const readLine: Effect.Effect<string> = Effect.succeed("OK")

const printLine = (message: string): Effect.Effect<void> =>
  Console.log(message)

const program = pipe(
  readLine,
  Effect.map(str => `You entered: ${str}`),
  Effect.tap(message => printLine(message))
)
```

```
// io.test
import { Effect } from "effect";
import { program } from "./io";

describe("io", () => {
  it("should be able to console log", () => {
    const result = Effect.runSync(program);
    expect(result).toEqual('You entered: OK');
  });
});
```

Lab 2 : IOEither

```ts
import { Effect, Console } from "effect"
import { right } from "effect/Either"
import { pipe } from "effect/Function"

// In this lab. IOEither wrap Effect because we want Error on the `left`, and right on the `right`
export type IOEither<E, A> = Effect.Effect<A, E>

const printLine = (message: string): IOEither<never, void> => Console.log(message)
const printHello: IOEither<never, void> = printLine('Hello')

export const program = (readLine: IOEither<string, string>) =>
  pipe(
    printHello,
    Effect.flatMap(() => readLine),
    Effect.flatMap((str) => right(`You entered: ${str}`)),
    Effect.match({
      onFailure: () => 'Error',
      onSuccess: (message) => message
    })
  )
```

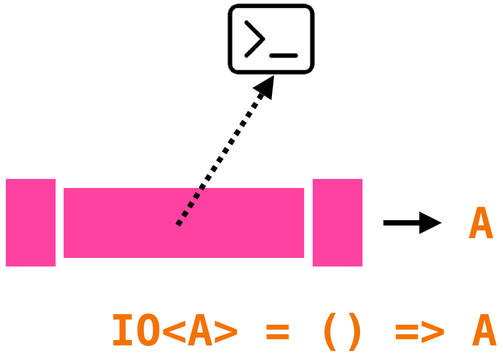
```ts
import { Effect } from "effect"
import { IOEither, program } from "./ioEither"

describe("ioEither", () => {
  const rightReadLine: IOEither<string, string> = Effect.succeed('OK')
  const leftReadLine: IOEither<string, string> = Effect.fail('Error')

  it("should be able to read string from console and print it", () => {
    expect(
      Effect.runSync(program(rightReadLine))
    ).toEqual('You entered: OK')
  })

  it("should be able to read string from console", () => {
    expect(Effect.runSync(program(leftReadLine))).toEqual('Error')
  })
})
```

Task

Compose asynchronous computations with Side Effects

Revisit IO Structure

```
type IO<A> = () => A
```



- **IO** model computational

  IO wrap generic type constructor over any type ‘A’. It’s a computational wrapped in a box so that we can pass it around as data. And return value based on type it wrapped. This function doesn’t receive and input and always return a value of type ‘A’.

- **IO** has side effects

  It can get or set values outside of its body.

- **IO** has no exception

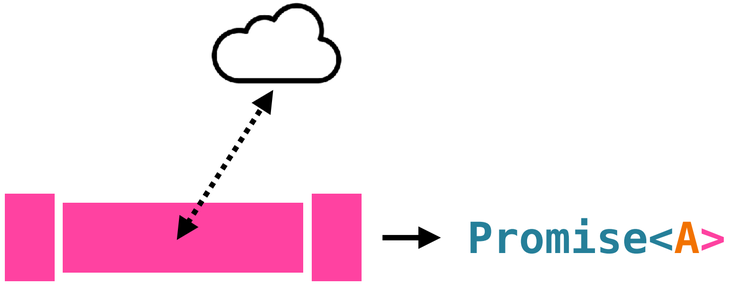
  When execute IO. It’s doesn’t error out.

- **IO** is synchronous

  It should be synchronous. When we execute the IO, it will block the execution until it’s done.

Asynchronous in TypeScript

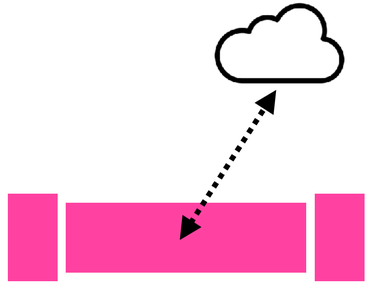
What if we want Asynchronous IO?




```
function fetchBooks(): Promise<Book[]> {
  return fetch("https://jsonplaceholder.typicode.com/posts")
    .then(res => res.json())
    .then(data => data as Book[])
}
```

IO with Asynchronous

What if we want Asynchronous IO?



→ `Promise<A>`

`IO<Promise<A>>`

```
function fetchBooks(): Promise<Book[]> {
  return fetch("https://jsonplaceholder.typicode.com/posts")
    .then(res => res.json())
    .then(data => data as Book[])
}
```

we use Promise to handle asynchronous here and wrap side effect with IO

```
function fetchBooks(): IO<Promise<Book[]>> {
  return () => fetch("https://jsonplaceholder.typicode.com/posts")
    .then(res => res.json())
    .then(data => data as Book[])
}
```

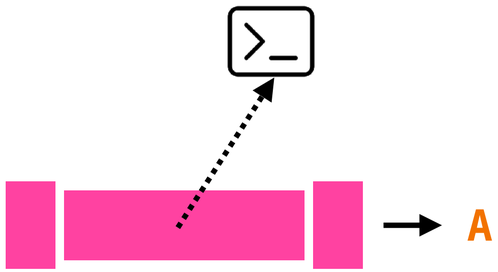
Can we just merge IO and Promise into a new structure? Yes. We call Task.

Task

Handle side effect asynchronously

We can simplify `IO<Promise<A>>` ***with*** *Task*

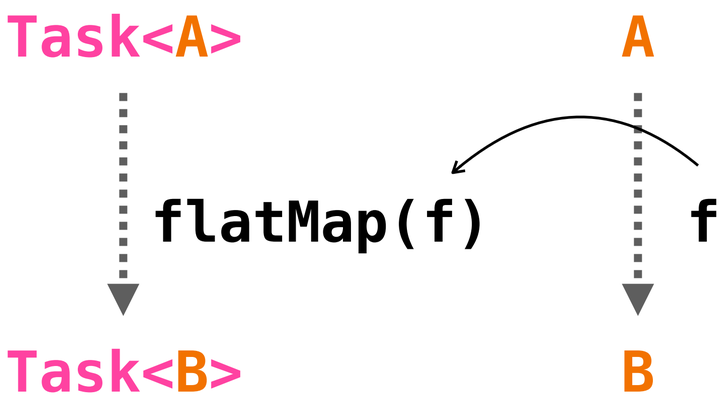
- ***Task*** *model computational*
- ***Task*** *has side effects*
- ***Task*** *has no exceptions*
- ***Task*** *is* ***Asynchronous***




```
Task<A> = () => IO<Promise<A>>
```

Task as a Functor

Task that can be map from type to type



- We can work with Task like IO, Either, Option and all structures we've seen before.
- We apply operations on Task using existing TypeClass operation it supports.
- We can have a map (flatMap) function for Task because Task is a Functor.

Task with Error handling

Task<Either<E, A>>

- By definition, Task should never fail or throw error.
- If you want to handle error due to side effect cannot be trusted. We can use Task to wrap Either.
- Instead of define `Task<Either<E,A>>`. We can define a dedicated structure called `TaskEither<E, A>`.

Lab 1 : TaskEither

fp-ts

```
import { pipe } from "fp-ts/function"
import * as TE from "fp-ts/TaskEither"

type User = {
  id: number
  name: string
  email: string
}

const fetchResponse = (url: string): TE.TaskEither<string, Response> =>
  TE.tryCatch(
    () => fetch(url),
    () => "Network error"
  )

const checkStatus = (res: Response): TE.TaskEither<string, Response> =>
  res.ok
    ? TE.right(res)
    : TE.left(`HTTP error: ${res.status}`)

const parseJson = (res: Response): TE.TaskEither<string, unknown> =>
  TE.tryCatch(
    () => res.json(),
    () => "Invalid JSON"
  )

const fetchUsers = (url: string): TE.TaskEither<string, User[]> =>
  pipe(
    fetchResponse(url),
    TE.flatMap(checkStatus),
    TE.flatMap(parseJson),
    TE.map((data) => data as User[])
  )

export const requestUsers: TE.TaskEither<string, User[]> = pipe(
  fetchUsers("https://jsonplaceholder.typicode.com/users"),
  TE.map((users) => users.map(u => ({ id: u.id, name: u.name, email: u.email }))),
  TE.orElseW((err) => TE.left(err))
)

// requestUsers().then(console.log)
```

Lab 2 : TaskEither

Effect

```
import { Effect } from "effect"

type User = {
  id: number
  name: string
  email: string
}

const fetchResponse = (url: string) =>
  Effect.tryPromise({
    try: () => fetch(url),
    catch: () => "Network error"
  })

const checkStatus = (res: Response) =>
  res.ok
    ? Effect.succeed(res)
    : Effect.fail(`HTTP error: ${res.status}`)

const parseJson = (res: Response) =>
  Effect.tryPromise({
    try: () => res.json(),
    catch: () => "Invalid JSON"
  })

const fetchUsers = (url: string) =>
  fetchResponse(url).pipe(
    Effect.flatMap(checkStatus),
    Effect.flatMap(parseJson),
    Effect.map((data) => data as User[])
  )

export const requestUsers =
  fetchUsers("https://jsonplaceholder.typicode.com/users").pipe(
    Effect.map(users => users.map(u => ({ id: u.id, name: u.name, email: u.email }))),
    Effect.catchAll(err => Effect.succeed({ error: err }))
  )

// Effect.runPromise(requestUsers).then(console.log)
```

Lab 3 : TaskEither (Optional)

Fetch JSON data example

```
import { tryPromise, catchAll, succeed, runPromise } from "effect/Effect"

type CryptoSnapshotItem = {
  coin: string
  priceUsd: number
  fetchedAt: string
}

const DEFAULT_COINS = ["bitcoin", "ethereum", "solana"]
const DEFAULT_VS_CURRENCY = "usd"

const urlWith = (coins: string[]) =>
  (vsCurrency: string): string =>
    `https://api.coingecko.com/api/v3/simple/price?ids=${coins.join(",")}&vs_currencies=${vsCurrency}`

const fetchSnapshot = (url: string) =>
  tryPromise({
    try: () => fetch(url)
      .then(res => res.json() as Promise<Record<string, { usd: number }>>)
      .then(data => Object.entries(data).map(toCryptoSnapshotItem)),
    catch: (error) => String(error),
  })

const toCryptoSnapshotItem = ([coin, { usd }]: [string, { usd: number }]): CryptoSnapshotItem => ({
  coin,
  priceUsd: usd,
  fetchedAt: new Date().toISOString(),
})

fetchSnapshot(urlWith(DEFAULT_COINS)(DEFAULT_VS_CURRENCY))
  .pipe(catchAll(err => succeed({ error: err })))
  .pipe(runPromise)
  .then(console.log)
```

Thank you

See you tomorrow